QATIPv3 Azure Lab4
Terraform variables and workspaces

Contents

Lab Objectives

In this lab you will:

- Deploy resources using a hard-coded terraform file
- Utilize variables to make your code reuseable
- Implement overrides using the command line and tfvars files
- Utilize terraform workspaces to allow multiple simultaneous deployments

Teaching Points

When writing terraform code, it is instinctive to supply explicit values when they are required, for example **name = "michael"**. This can make the code very static though, requiring manual changes of these values each time a modification is needed. A better practice is to use 'placeholders', variables, to which values can be passed at runtime, for example **name = var.username**. Passing values to these variables can be achieved using variable file default values, tfvars files, command line supplied or by being prompted. By default, all changes made to your configuration will be tracked by a single state file. Workspaces allow multiple

state files to exist simultaneously so that the same base code, provided with unique variable values, can be deployed. This makes your code flexible and reuseable.

Before you begin

1. Ensure you have completed Lab0 before attempting this lab.
2. In the IDE terminal pane, enter the following command…

   cd c:\azurelabs\lab4
3. This shifts your current working directory to lab4. Ensure all commands are executed in this directory
4. Close any open files and use the Explorer pane to navigate to and open the lab4 folder.

Solution

There is no solution code for this lab as it involves multiple deployment phases. Reach out to your instructor if you encounter issues.

Task1- Review provisioned terraform files

1. Update line 12 of **main.tf** with your lab subscription id.
2. Update line 17 of **main.tf** with your resource group Suffix
3. Review the updated **main.tf** file.

   There is a hard coded definition of a resource group in North Europe. This represents the resources we are to create, in this case just an empty resource group. In production there would be many other resources defined here. There is also an output block which displays the current workspace.

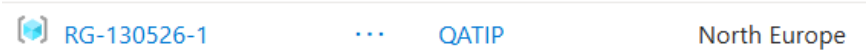
Task2- Run terraform plan and apply with hardcoded parameter values

1. Ensure you have navigated to the **c:\azurelabs\lab4** folder
2. Run **terraform init**
3. If required, run **az login** to authenticate to azure
4. Run **terraform plan**
5. Review the plan output, your resource group name will differ..

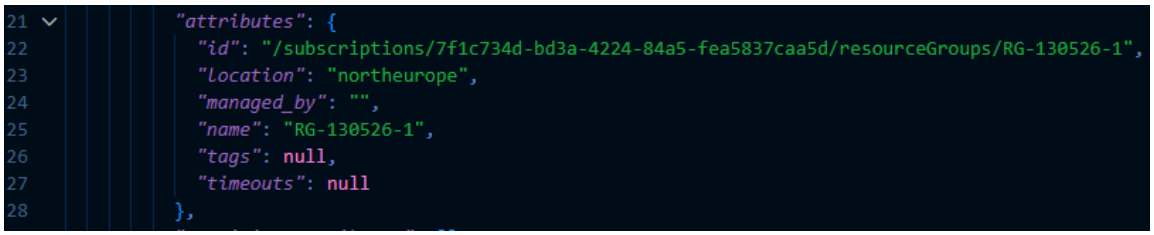
```
Terraform will perform the following actions:

  # azurerm_resource_group.example will be created
  + resource "azurerm_resource_group" "example" {
      + id       = (known after apply)
      + location = "northeurope"
      + name     = "RG-130526-1"
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + workspace_name = "default"
```

6. Run **terraform apply** followed by **yes** when prompted

7. Switch to the Azure portal and verify the creation of your resource group in North Europe. Others may also exist…



8. Examine the newly created state file in the root folder..



9. **Takeaway**: Hardcoded parameter values will always be used if they exist. This can make the code inflexible and static

Task3- Substitute hardcoded parameter vales with variables

1. In **main.tf**; Replace hard-code resource group name with **var.resource_group_name** (no quotes)

2. Replace hard-coded location "**northeurope**" with **var.location**

```
# Resource Group
resource "azurerm_resource_group" "example" {
  name     = var.resource_group_name
  location = var.location
}
```

Note that, with terraform extensions installed, the IDE will flag the problem of variables being referenced but not declared if you hover over the variables.

3. Run **terraform plan** and review the errors...

```
PS C:\azurelabs-new\lab4> terraform plan

Error: Reference to undeclared input variable

  on main.tf line 17, in resource "azurerm_resource_group" "example":
  17:   name     = var.resource_group_name

An input variable with the name "resource_group_name" has not been declared. This variable can be declared with a variable "resource_group_name" {} block.


Error: Reference to undeclared input variable

  on main.tf line 18, in resource "azurerm_resource_group" "example":
  18:   location = var.location

An input variable with the name "location" has not been declared. This variable can be declared with a variable "location" {} block.
```

4. **Takeaway**: If variables are referenced in your code, then they **must** be declared.
5. In **variables.tf**; uncomment all lines except lines 4 and 10. This declares the variables but does not provide default values...

```
1   variable "resource_group_name" {
2     description = "The name of the Azure Resource Group"
3     type        = string
4   #  default     = "RG-{Suffix}"
5   }
6
7   variable "location" {
8     description = "The Azure region to deploy the Resource Group"
9     type        = string
10  #  default     = "North Europe"
11  }
```

6. Run **terraform plan**
7. When prompted, enter **northeurope** as the location and **RG-{Suffix}** as the resource group name..

```
PS C:\azurelabs-new\lab4> terraform plan
var.location
  The Azure region to deploy the Resource Group

  Enter a value: northeurope

var.resource_group_name
  The name of the Azure Resource Group

  Enter a value: RG-130526-1

azurerm_resource_group.example: Refreshing state... [id=/subscriptions/7f1c73

No changes. Your infrastructure matches the configuration.
```

The planning completes using the values inputted when prompted. Given that these values are the same as the old static values, the planning phase shows that there are no changes needed…

8. In **variables.tf**; uncomment lines 4 and 10 and update line 4 with your suffix
9. Run **terraform plan**
10. The planning completes using the values drawn from the variables file. Given that these values are the same as the old static values, the planning phase shows that there are no changes needed…

```
No changes. Your infrastructure matches the configuration.
```

11. In **variables.tf**; append a random digit to your resource group name.
12. Run **terraform plan**
13. The planning completes using the values drawn from the variables file. Given that the **name** values has changed, terraform flags that the existing resource group would be destroyed, and a new one created if we were to apply this new configuration..

```
Plan: 1 to add, 0 to change, 1 to destroy.
```

14. In **variables.tf**; remove the random digit from your resource group default name
15. Destroy the current deployment using **terraform destroy** followed by **yes**

Takeaway: If a variable is declared but no value has been assigned then you are prompted for values. If a variable is declared with a default value then this value will be used unless overridden. All changes to variable values will apply to the current deployment as you are working in the default workspace. Workspaces allow multiple deployments to exist simultaneously, all using the same code but with different variable values applied as needed. Workspaces will be introduced later in this lab.

Task4- Override variable values at the command prompt

1. Enter the following command..
   terraform plan -var="location=westus"

```
PS C:\azurelabs-new\lab4> terraform plan -var="location=westus"

Terraform used the selected providers to generate the following e
  + create

Terraform will perform the following actions:

  # azurerm_resource_group.example will be created
  + resource "azurerm_resource_group" "example" {
      + id       = (known after apply)
      + location = "westus"
      + name     = "RG-130526-1"
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + workspace_name = "default"
```

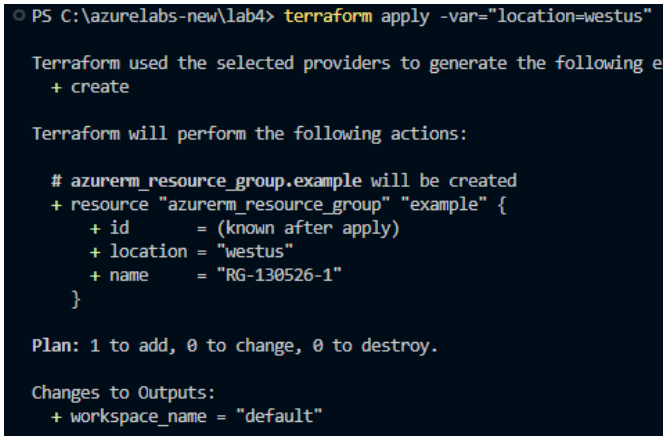
   Note that the location supplied at the command prompt takes precedence over the default location.

2. Run **terraform apply** and review the proposed actions…

```
PS C:\azurelabs-new\lab4> terraform apply

Terraform used the selected providers to generate th
  + create

Terraform will perform the following actions:

  # azurerm_resource_group.example will be created
  + resource "azurerm_resource_group" "example" {
      + id       = (known after apply)
      + location = "northeurope"
      + name     = "RG-130526-1"
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + workspace_name = "default"

Do you want to perform these actions?
  Terraform will perform the actions described above
  Only 'yes' will be accepted to approve.

  Enter a value:
```

   Note that location has reverted back to the default **northeurope** because an override value was not provided at the command line.

3. Press enter, **without typing yes** to cancel the apply
4. Run **terraform apply -var="location=westus"** and note that the apply uses the specified location…



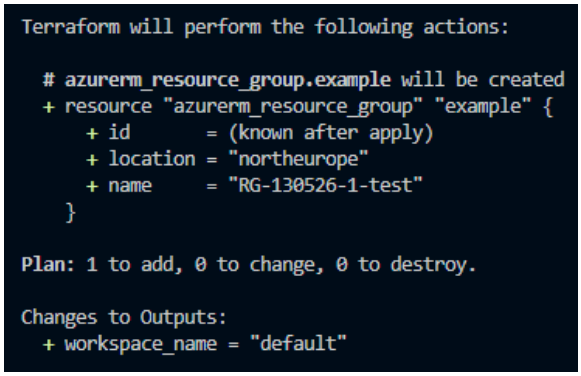
5. Press enter, **without typing yes** to cancel the apply

Takeaway: Supplying a variable value at the command prompt using **-var=" "** has the highest priority and overrides values supplied ***anywhere*** else. In this case **westus** was used as the **location** value as it was entered at the command prompt, whilst the **name** value was drawn from the variables file. If there was no default value then you be prompted.

If using command line variables then they **must** be used consistently for **both** planning and applying but they are not needed for destroy actions.

Task5- Utilize terraform.tfvars to supply variable values

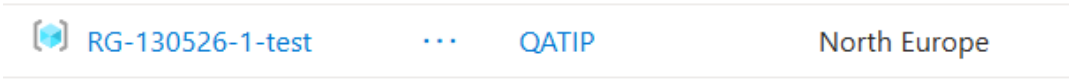
1. In **terraform.tfvars**; Uncomment line 1 and update the resource group name with your suffix. Note that this resource group name differs from that in the variables.tf file
2. Run **terraform plan**



3. **Takeaway**: terraform.tfvars values, if they exist, are loaded automatically and take precedence over default values. Therefore, the resource group name is drawn from terraform.tfvars and the location is drawn from variables.tf

4. In **terraform.tfvars**; uncomment line 2
5. In variables.tf; comment the lines that supply the default values for the two variables.
6. Run **terraform plan**

   The plan is identical but now all variable values are drawn from terraform.tfvars. If terraform.tfvars provides a value then defining a default value when the variable is declared becomes redundant as terraform.tfvars has higher precedence.
7. Run **terraform apply** followed by **yes**
8. Switch to the Azure portal and verify the creation of resource group in North Europe..



Takeaway: When terraform.tfvars supplies **all** variable values, there is no rationale for specifying default values as these will always be overridden. It is not uncommon therefore to define the variables without declaring any default values.

Task6 Named tfvars and Terraform Workspaces

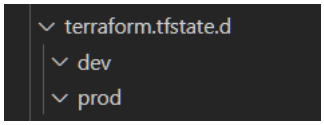
1. Create two workspaces; "**dev**" and "**prod**"…
   terraform workspace new dev
   terraform workspace new prod
   terraform workspace list

```
Created and switched to workspace "dev"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\azurelabs\lab4> terraform workspace new prod
Created and switched to workspace "prod"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\azurelabs\lab4> terraform workspace list
  default
  dev
* prod
```

The * indicates your current workspace is **prod**.

The **default** workspace always exists, and this is where your current deployment of the 'test' resource group to North Europe is tracked by the statefile **terraform.tfstate** in the root folder.

2. In EXPLORER, note the creation of a new folder "**terraform.tfstate.d**" with an empty subfolder for each of the new workspaces…

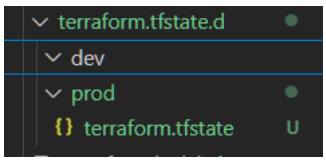


3. Update **prod.tfvars** with your Suffix, thus defining a 'prod' resource group in UK South.
4. Run **terraform plan --var-file=prod.tfvars**

```
Terraform will perform the following actions:

  # azurerm_resource_group.example will be created
  + resource "azurerm_resource_group" "example" {
      + id       = (known after apply)
      + location = "uksouth"
      + name     = "RG-130526-1-prod"
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + workspace_name = "prod"
```

You are working in the **prod** workspace, and the resource group name and location values are drawn from **prod.tfvars**

5. Run **terraform apply --var-file=prod.tfvars** and enter **yes**



Note that a new **terraform.tfstate** file is created in the **prod** folder to track the prod workspace deployment

6. Switch to the Azure portal and verify the creation of your prod resource group in West Europe...

RG-130526-1-prod	···	QATIP	UK South
RG-130526-1-test	···	QATIP	North Europe

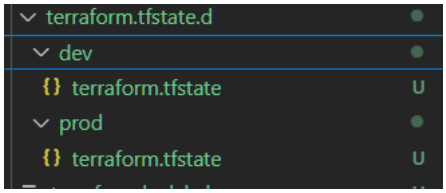
Note that your test resource group in North Europe still exists as it is a separate deployment

7. Switch to the dev workspace..
   terraform workspace select dev
8. Update **dev.tfvars** with your Suffix, thus defining a 'dev' resource group in West Europe.
9. Run **terraform plan --var-file=dev.tfvars**

```
Terraform will perform the following actions:

  # azurerm_resource_group.example will be created
  + resource "azurerm_resource_group" "example" {
      + id       = (known after apply)
      + location = "westeurope"
      + name     = "RG-130526-1-dev"
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + workspace_name = "dev"
```

You are working in the **dev** workspace, and the resource group **name** and **location** values are drawn from **dev.tfvars**

10. Run **terraform apply --var-file=dev.tfvars** followed by **yes**



Note that a new terraform.tfstate file is created in the **dev** folder to track the dev workspace deployment

11. Switch to the Azure portal and verify the creation of dev resource group in West Europe ..

RG-130526-1-test	···	QATIP	North Europe
RG-130526-1-prod	···	QATIP	UK South
RG-130526-1-dev	···	QATIP	West Europe

Note that your previously deployed resources still exist as they are separate deployments.

Takeaway: tfvars files other than terraform.tfvars must be applied at the command line using **--var-file=<name of tfvars file>**. Values declared in these files take precedence over terraform.tfvars values if they conflict. Note: If multiple tfvars are applied, then the last loaded has highest priority if there are conflicts. There is no mandatory correlation between the name of the workspace and the name of the tfvars file used in it.

Workspaces allow the same base code to be used multiple times with varying variable values being applied through the use of named tfvars files. Each workspace creates its own statefile, allowing multiple deployments to exist simultaneously. This is typically used to deploy the same resources across different environments.

Task7- Lab Clean-up

1. When deleting resources in workspaces, always pay close attention to workspace you are currently working in. Use **terraform workspace show** to determine your current workspace..

```
PS C:\azurelabs\lab4> terraform workspace show
dev
PS C:\azurelabs\lab4>
```

2. Destroy the resources in the current workspace (dev)...

   terraform destroy

   Note that whilst tfvars files **must** be specified performing plan and apply actions, they are not needed for destroy actions

3. Enter **yes** when prompted

4. The current workspace cannot be deleted so move to the **prod** workspace and delete the **dev** workspace…

   terraform workspace select prod
   terraform workspace delete dev

5. Destroy the resources in the current workspace (prod) using..
   terraform destroy and enter **yes** when prompted

6. Move to the **default** workspace and delete the **prod** workspace…

terraform workspace select default
terraform workspace delete prod

7. Verify the deletion of the workspaces, leaving just the default workspace. This cannot be deleted..

   terraform workspace list

```
PS C:\azurelabs\lab4> terraform.exe workspace list
  * default
```

8. Note that the workspace directories are deleted along with the workspaces themselves…

9. Destroy the resources in the current workspace (default) using..
   terraform destroy

10. Enter **yes** when prompted

11. Switch to the Azure portal to confirm the deletion of all three resource groups.

Congratulations, you have completed this lab